

Multi Agent Workflow(Maths Agent and Knowledge Agent):

1.Initially developed state class which inherits from TypedDict. Here messages are concantenated for messages and user_input is latest user query and messages for all chat messages.

```
✓ class multiagent(TypedDict):  
    messages : Annotated[Sequence[BaseMessage], operator.add]  
    user_input : str
```

2. Utilized the llama3.2:latest as llm fom ChatOllama.

```
✓ llm = ChatOllama(  
    |     model="llama3.2:latest"  
    )
```

3.Router and Nodes:

3.1-Router:

router function:

The router is made as entry point which will decide which agent should handle the query.If user input has math related math-related words ,expressions or digits it will redirect to math agent or will be routed to knowledge agent.

```
✓ def router(state:multiagent)->str:  
    """Routes to either math_agent or knowledge_agent based on the user input """  
    latest_message = state['messages'][-1].content.lower()  
    ✓ if any(char.isdigit() for char in latest_message) or any(  
        | opr in latest_message for opr in ['+', '-', '*', '%', '/', 'add', 'subtract', 'multiply', 'divide', 'calculate', 'equation', 'exp  
        | ):  
        |     return "math_agent"  
    |     return "knowledge_agent"
```

3.2-Nodes:

math_calculation_node: Takes state["user_input"] and chat history as input and returns AIMessage as "Math Agent".

```
✓ def math_calculation_node(state:multiagent)->dict:  
    """Makes mathematical calculations"""  
    chat_history = "\n".join([msg.content for msg in state["messages"]])  
    prompt = f"You are a Mathematical assistant who answers mathematical questions clearly and precisly.. . Here is the previous conv  
    output= llm.invoke(prompt).content  
    return {'messages':[AIMessage(content=output, additional_kwargs={"agent":"Math Agent"})]}
```

knowledge_node: Handles general/factual questions that takes state["user_input"] and chat history as input and returns AIMessage as "Knowledge Agent".

```

def knowledge_node(state:multiagent)->dict:
    """Makes Knowledge based answers"""
    chat_history = "\n".join([msg.content for msg in state["messages"]])
    prompt = f"You are a knowledgeable assistant who answers factual, general, and conversational questions clearly and precisely.. He
    output= llm.invoke(prompt).content
    return {"messages":[AIMessage(content=output, additional_kwargs={"agent":"Knowledge Agent"})]}

```

And dummy node is required as langgraph entry point-router_passthrough.

4.Langgraph workflow :

Starts at **router** node. Routes to either **math_calculation_node** or **knowledge_node** and ends (END node) execution after generating response. After building the workflow the show_graph method is called which imports from langchain_util.py file which has logic to show compiled graph.

```

graph = StateGraph(multiagent)
graph.add_node("math_calculation_node", math_calculation_node)
graph.add_node("knowledge_node", knowledge_node)
graph.add_node("router", router_passthrough)
graph.set_entry_point("router")
graph.add_conditional_edges(
    "router",
    router,
    {"math_agent": "math_calculation_node", "knowledge_agent": "knowledge_node"},
)
graph.add_edge("math_calculation_node", END)
graph.add_edge("knowledge_node", END)

```

langchain_util.py :

```

from PIL import Image
from io import BytesIO
from langchain_core.runnables.graph import MermaidDrawMethod

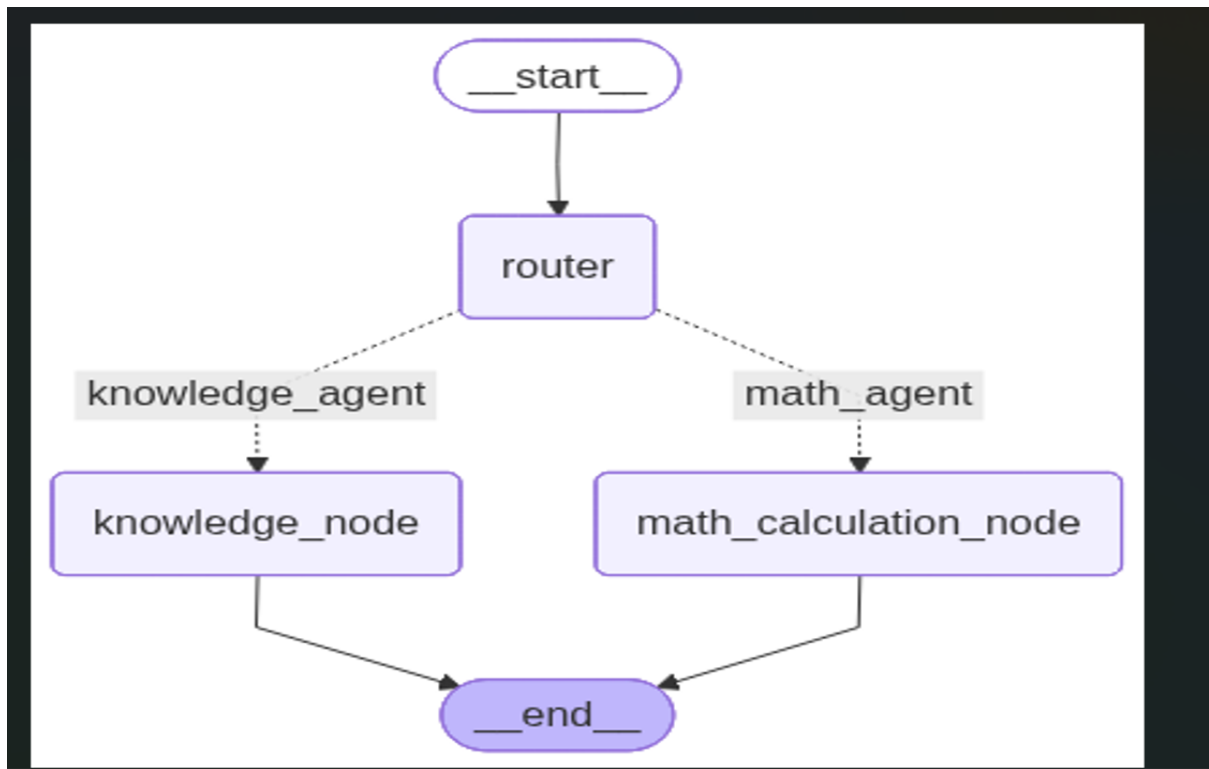
def show_graph(runnable):

    graph_image = runnable.get_graph().draw_mermaid_png(
        draw_method=MermaidDrawMethod.API, # Always online API
        output_file_path="../graph.png"
    )

    img = Image.open(BytesIO(graph_image))
    img.show()

```

The compiled graph is shown below which will be utilised for executions.



5.Memory handling:

Once the langraph workflow is built ,proceeded with memory. The thread_id is generated from uuid which generated unique id for every session.

```
✓ if "thread_id" not in st.session_state:
    |     st.session_state.thread_id = str(uuid.uuid4())
✓ if "messages" not in st.session_state:
    |     st.session_state.messages = []
```

MemorySaver is used which ensures the state remains persistent during a session.

```
checkpointer = MemorySaver()
if "user_input" not in st.session_state:
    |     st.session_state.user_input = ""
user_query = st.text_area("Ask a question:",key="user_input")
workflow=graph.compile(checkpointer)
show_graph(workflow)
```

6. Streamlit:

6.1- User enters text in Streamlit text area and input is stored in `st.session_state["user_input"]`. When user clicks Send button the new `HumanMessage` is added to message list.

6.2-The router executes and detects whether to route for `math_agent` or `knowledge_agent` based on user input. Corresponding agent will generate the output.

6.3-The output is saved in the session which will be persistent across the workflow run.

Streamlit UI shows latest response and also which is responding based on input. The Reset button will clear the state and restarts the session.

Knowledge agent is responding for general queries.

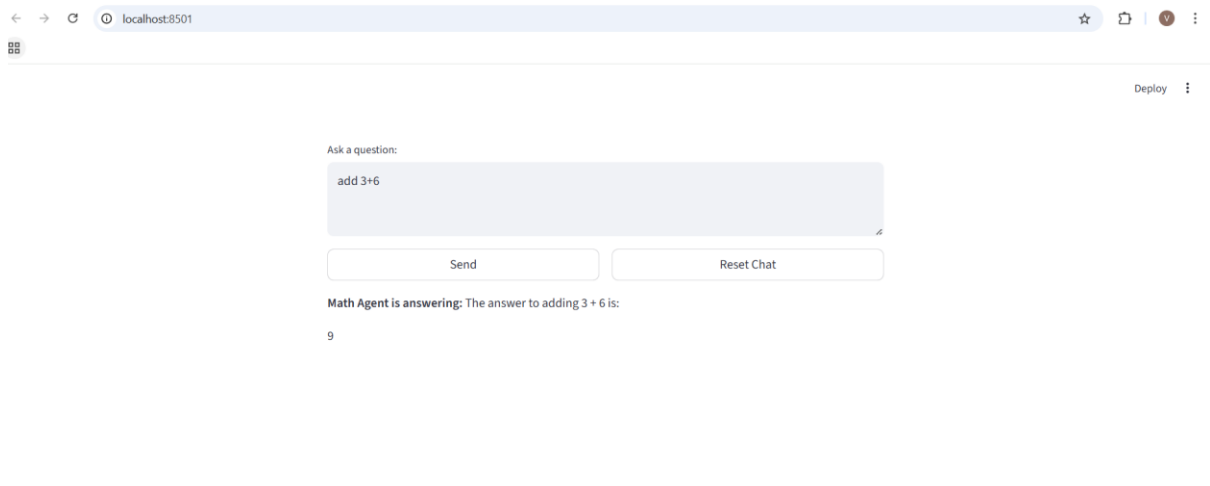
```
✓ if reset_clicked:
✓     for key in ["messages", "thread_id", "user_input"]:
✓         if key in st.session_state:
✓             del st.session_state[key]
✓     st.session_state.thread_id = str(uuid.uuid4())
✓     st.rerun()

✓ if send_clicked and user_query:
✓     config = {"configurable": {"thread_id": st.session_state.thread_id}}
✓     updated_messages = st.session_state.messages + [HumanMessage(content=user_query)]
✓     state = {
✓         "messages": updated_messages,
✓         "user_input": user_query
✓     }
✓     output = workflow.invoke(state, config=config)
✓     st.session_state.messages = output["messages"]

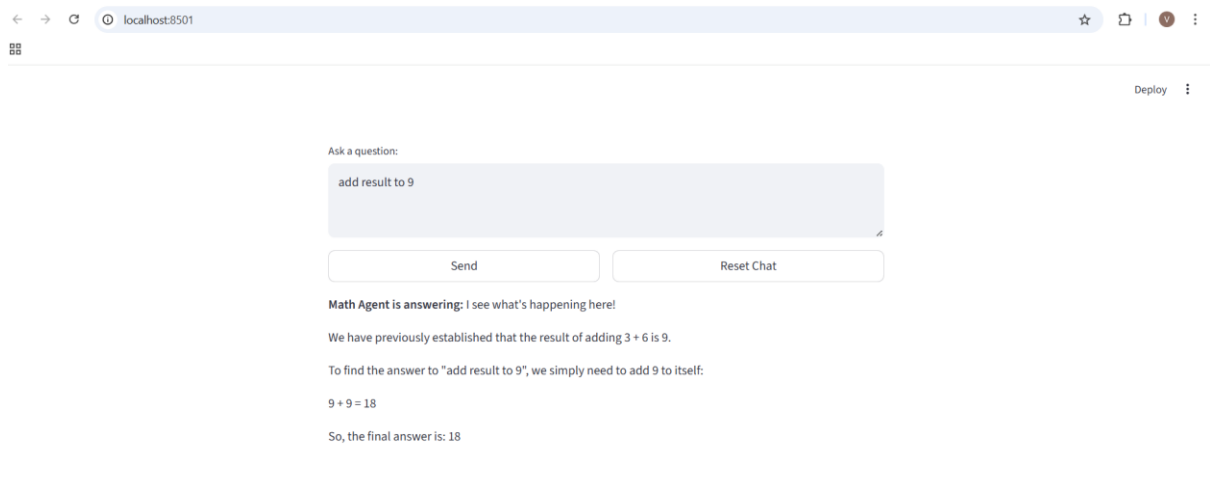
✓ if st.session_state.messages:
✓     last_msg=st.session_state.messages[-1]
✓     if isinstance(last_msg, AIMessage):
✓         agent = last_msg.additional_kwargs.get("agent", "AI")
✓         st.markdown(f"**{agent} is answering:** {last_msg.content}")
```



Maths agent is responding for mathematical questions.



The previous numbers are also remembered and the mathematical operations are done.



The **Reset** button is clicked and text area is cleaned and new thread_id is assigned.

